Technical Appendix for *scd* Package in R

Chun Pang Chow, Ratza Rincón and Kyungchul Song

This appendix establishes the mathematical equivalence between the theoretical estimators defined in Section 5 of Rincón and Song (2025) and the Computational implementation provided in the *scd* package in R. While the theory is presented using standard algebraic definitions, the software employs specific matrix decompositions and algorithmic optimizations to ensure numerical stability and to accommodate survey weights. We detail these mappings below.

1 Estimation of Weights for Donors and the Target Parameter

1.1 Theoretical object

Synthetic Control with Differencing (SCD) estimates the vector of weights for the donors $\hat{w}$ as the minimizer of the sum of squares of matching errors during the pre-treatment:

$$\hat{w} = \arg\min_{w \in \Delta_{K-1}} \sum_{t \in \mathcal{T}_0} \left(\hat{\mu}_{0,t}(\lambda) - \sum_{j=1}^{K} \hat{\mu}_{j,t}(\lambda) w_j \right)^2, \tag{1}$$

where $\hat{\mu}_{j,t}(\lambda)$ is the within-group-differenced mean of group j at time t, and $\lambda \in \Delta_{|\mathcal{T}_0|-1}$ is the differencing parameter. To keep notation simple, we omit the dependency on λ when considering $\hat{\mu}_{j,t}$. Using these weights, the SCD method estimates the average treatment effect on the target group at period t as follows

$$\hat{\theta}_t(\lambda, \hat{w}) = \hat{\mu}_{0,t} - \sum_{j=1}^{K} \hat{\mu}_{j,t} \hat{w}_j. \tag{2}$$

Identification of the target parameter and consistency of the estimator are discussed in the paper.

1.2 Computational implementation

The function `att_scd.R` first computes the group-specific weighted sample means as follows

$$\begin{aligned} \hat{m}_{j,t} &= \frac{\sum_{i=1}^{n} \pi_{i,t} Y_{i,t} 1\{G_{i,t} = j\}}{\sum_{i=1}^{n} \pi_{i,t} 1\{G_{i,t} = j\}} \\ &= \frac{1}{\hat{n}_{j,t}} \sum_{i \in N_{j,t}} \pi_{i,t} Y_{i,t}, \end{aligned}$$

where $\pi_{i,t}$ denotes the survey weight of observation i at period t, and $\hat{n}_{j,t}$ is the sum of weights for individuals in $N_{j,t} = \{i \in N_t : G_{i,t} = j\}$. In the code:

```
# Weighted group means
means_ordered <- collap(data, Y ~ t + G, w = ~ weight)
```

The within-group-differenced means $\hat{\mu}_{j,t}$ are constructed using the weighted means $\hat{m}_{j,t}$ and form the $(T^* - 1) \times 1$ vector

$$\hat{\gamma}_{pre} = [\hat{\mu}_{0,1}, ..., \hat{\mu}_{0,T^*-1}]',$$

and the $(T^* - 1) \times K$ matrix

$$\hat{\Gamma}_{pre} = [\hat{\boldsymbol{\mu}}_1 \ \ldots \ \hat{\boldsymbol{\mu}}_{T^*-1}]',$$

where $\hat{\boldsymbol{\mu}}_t = [\hat{\mu}_{1,t}, ..., \hat{\mu}_{K,t}]'$. A similar vector and matrix are created for the post-treatment period. The objective function is thus implemented as:

```
diff_vec <- hat_gamma_pre - hat_Gamma_pre %*% w
objective = crossprod(diff_vec)
```

The `crossprod` function computes the inner product, expanding to $\sum_{t=1}^{T_0}(\hat{\mu}_{0,t} - \sum_{j=1}^{K} \hat{\mu}_{j,t} w_j)^2$. We use the sequential quadratic programming algorithm NLOPT_LD_SLSQP from the *nloptr* package in R to solve the constraint optimization problem in (1), where the vector of starting values is specified as $w_0 = (1/K, ..., 1/K) \in \mathbf{R}^K$. In the code:

```
obj_func <- function(w) {
    diff_vec <- hat_gamma_pre - hat_Gamma_pre %*% w
    list(objective = crossprod(diff_vec), gradient = -2 * crossprod(hat_
       Gamma_pre, diff_vec))
}
eq_constr <- function(w) list(constraints = sum(w) - 1, jacobian = rep(1,
   length(w)))

opt_res <- nloptr::nloptr(
    x0 = rep(1/K, K), eval_f = obj_func, eval_g_eq = eq_constr,
    lb = rep(0, K), ub = rep(1, K),
    opts = list(algorithm = "NLOPT_LD_SLSQP", xtol_rel = 1e-10, ftol_abs =
        1e-14, maxeval = 100000, print_level = 0)
)

hat_w <- round(opt_res$solution, wdecimals)
```

Lastly, the point estimate for the target parameter is obtained as

$$\hat{\theta}(\lambda, \hat{w}) = \hat{\gamma}_{post} - \hat{\Gamma}_{post}\hat{w}. \tag{3}$$

In the code:

```
hat_theta_scd <- hat_gamma_post - hat_Gamma_post %*% hat_w
```

2 Asymptotic Variance of $B_2' \, \hat{\varphi}(w)$ under Repeated Cross-Sections

2.1 Theoretical object

In repeated cross-sectional data the asymptotic variance of $\sqrt{n}B_2'(\hat{\varphi}(w) - \varphi(w))$ is estimated as

$$\hat{V}_{RC}(w) = \frac{1}{T^* - 1} \sum_{t=1}^{T^*-1} \left[\frac{1}{n} \sum_{i \in N} \hat{V}_{i,t}(w)^2 \times B_2' \mathbf{M}_t B_2 \right], \tag{4}$$

where

$$\hat{V}_{i,t}(w) = \sum_{j=1}^{K} w_j \hat{\psi}^*_{ij,t} - \hat{\psi}^*_{i0,t} \text{ and },$$

and

$$\hat{\psi}^*_{ij,t} = \pi_{i,t} \frac{n}{\hat{n}_{j,t}} 1\{G_{i,t} = j\}(Y_{i,t} - \hat{m}_{j,t}),$$

and

$$\mathbf{M}_t = \frac{\hat{\boldsymbol{\mu}}_t \hat{\boldsymbol{\mu}}_t'}{T^* - 1} - \lambda_t(\bar{\boldsymbol{\mu}}\hat{\boldsymbol{\mu}}_t' + \hat{\boldsymbol{\mu}}_t\bar{\boldsymbol{\mu}}') + (T^* - 1)\lambda_t^2 \bar{\boldsymbol{\mu}}\bar{\boldsymbol{\mu}}'.$$

Here, B_2 is the $K \times (K-1)$ matrix of eigenvectors corresponding to the non-zero eigenvalues of the centering matrix $I_K - \frac{1}{K}\mathbf{1}\mathbf{1}'$, and $\bar{\boldsymbol{\mu}}$ is the pre-treatment average vector of $\hat{\boldsymbol{\mu}}_t$'s.

2.2 Computational implementation

The function `gen_hat_V_RC.R` computes $\hat{V}_{RC}(w)$ in three general steps within a for loop over t in the pre-treatment.

Step 1. We first compute $\frac{1}{n}\sum_{i \in N} \hat{V}_{i,t}(w)^2$ for a particular t. The code executes:

```
gen_hat_V_RC <- function(w, hat_Gamma_pre, data, lambda, B2, group_means,
   n_jt, K, n, T_star) {

  T_0 <- T_star - 1
  bar_mu <- colMeans(hat_Gamma_pre)
  weight_vec <- c(1, -w)  # treated = 1, donor k = -w[k]
  hat_V_sum <- matrix(0, nrow = K-1, ncol = K-1)

  for (pre_t in 1:T_0) {
    idx_t <- which(data$t == pre_t)
    if (length(idx_t) == 0) next

    G_idx <- data$G[idx_t] + 1L
    lin_idx <- G_idx + (pre_t - 1L) * (K + 1L)

    # psi*_{i,t} = (n / n_{j,t}) * (Y - m_{j,t})
    psi_star <- (n / n_jt[lin_idx]) * (data$Y[idx_t] - group_means[lin_idx
       ])

    # sum_j w_j psi*_{ij,t} - psi*_{i0,t}
    psi_wt <- psi_star * weight_vec[G_idx]

    # (1/n) sum_i pi_i^2 * psi_wt^2
    hat_V_it <- sum(data$weight[idx_t]^2 * psi_wt^2) / n
    ...
```

Step 2. Secondly, the script constructs the matrix $\mathbf{M}_t$ as follows:

```
    ...
    # M_t matrix
    mu_t <- hat_Gamma_pre[pre_t, ]
    M_t <- (1/T_0) * tcrossprod(mu_t) -
           lambda[pre_t] * (tcrossprod(bar_mu, mu_t) + tcrossprod(mu_t,
              bar_mu)) +
           T_0 * lambda[pre_t]^2 * tcrossprod(bar_mu)
    ...
```

Step 3. Lastly, the function accumulates in `hat_V_sum` the contribution for each pre-treatment period and returns the total sum of these terms normalized by the number of pre-treatment periods $T^* - 1$:

```
    ...
    hat_V_sum <- hat_V_sum + hat_V_it * crossprod(B2, M_t) %*% B2
  }
  return((1/T_0) * hat_V_sum)
}
```

3 Asymptotic Variance of $B_2' \, \hat{\varphi}(w)$ under Panel Data

When the data structure corresponds to a balanced panel, the observations are no longer independent across time. This induces cross-temporal covariances that must be included in the estimator of the asymptotic variance of $\sqrt{n}B_2'(\hat{\varphi}(w) - \varphi(w))$.

3.1 Theoretical object

In the balanced panel setting, we have $N_t = N$ for all $t \in \mathcal{T}$ and set the group-membership variable as time-invariant, i.e. $G_{i,t} = G_i$ for all $t \in \mathcal{T}$. In this case, the variance matrix is estimated as:

$$\hat{V}(w) = B_2' \left(\frac{1}{n} \sum_{i \in N} \hat{\boldsymbol{z}}_i(w) \hat{\boldsymbol{z}}_i'(w) \right) B_2, \tag{5}$$

where

$$\hat{\boldsymbol{z}}_i(w) = \sum_{j=1}^{K} w_j \hat{\boldsymbol{z}}_{ij} - \hat{\boldsymbol{z}}_{i0},$$

and

$$\hat{\boldsymbol{z}}_{ij} = \frac{1}{T^* - 1} \sum_{t=1}^{T^*-1} \hat{\boldsymbol{\mu}}_t \hat{\psi}_{ij,t},$$

and

$$\hat{\psi}_{ij,t} = \hat{\psi}^*_{ij,t} - \sum_{s \in \mathcal{T}_0} \lambda_s \hat{\psi}^*_{ij,s}.$$

3.2 Computational implementation

The functions `precompute_panel_all.R` and `gen_hat_V_panel_fast.R` compute $\hat{V}(w)$ in three steps.

Step 1. For each individual i, `precompute_panel_all.R` creates first the λ-differenced influence function at each period:

```
precompute_panel_all <- function(w, data, hat_Gamma_pre, lambda, B2,
                                 group_means, n_jt, K, n, T_star, T_total
                                    ) {

  T_0 <- T_star - 1

  data <- data[order(data$id, data$t), ]
  ids <- unique(data$id)
  N <- length(ids)

  # Compute psi* for ALL observations - O(n)
  # psi*_{i,t} = (n / n_{G_i,t}) * (Y_{i,t} - m_{G_i,t})
  G_vec <- data$G
  t_vec <- data$t
  Y_vec <- data$Y
  wt_vec <- data$weight
  G_idx <- G_vec + 1L

  lin_idx <- G_idx + (t_vec - 1L) * (K + 1L)
  psi_star <- (n / n_jt[lin_idx]) * (Y_vec - group_means[lin_idx])

  # Reshape to N x T_total matrices
  Psi_star_mat <- matrix(psi_star, nrow = N, ncol = T_total, byrow = TRUE)
  Wt_sq_mat <- matrix(wt_vec^2, nrow = N, ncol = T_total, byrow = TRUE)

  # Get group for each individual (first observation per individual)
  # Data is sorted by (id, t), so individual i's first obs is at row (i-1)
     *T_total + 1
  G_by_id <- G_vec[seq(1L, N * T_total, by = T_total)]
  wt_sq_ind <- Wt_sq_mat[, 1L]  # Individual weight (constant across t)


  # Compute DIFFERENCED psi for Z computation
  # psi_{i,t} = psi*_{i,t} - sum_s lambda_s * psi*_{i,s}
  Psi_pre <- Psi_star_mat[, 1:T_0, drop = FALSE]  # N x T_0

  # diff_term_z = sum_s lambda_s * psi*_{i,s} for each individual
  diff_term_z <- as.vector(Psi_pre %*% lambda)  # N x 1

  # Psi_diff[i,t] = psi*_{i,t} - diff_term_z[i] for pre-treatment periods
  Psi_diff_pre <- Psi_pre - diff_term_z  # Broadcasting: N x T_0
  ...
```

Step 2. The script then computes the vector $\boldsymbol{z}_i(w)$:

```
  ...
  # Compute Z_i for hat_V using DIFFERENCED psi
  # z_i = alpha_i * (1/T_0) sum_t mu_t * psi_{i,G_i,t}
  alpha_lookup <- c(-1, w)
  alpha_vec <- alpha_lookup[G_by_id + 1L]

  # Weight by alpha and compute Z_raw
  Psi_diff_weighted <- alpha_vec * Psi_diff_pre  # N x T_0

  # Z_raw = (1/T_0) * Psi_diff_weighted %*% hat_Gamma_pre
  # [N x T_0] * [T_0 x K] = [N x K]
  Z_raw <- (1 / T_0) * (Psi_diff_weighted %*% hat_Gamma_pre)

  # Transform to (K-1) space
  Z_B2 <- Z_raw %*% B2  # N x (K-1)
  ...
```

Step 3. Finally, the function `gen_hat_V_panel_fast.R` takes the output of function `gen_hat_V_panel_fast.R`, and creates the variance matrix.

```
  ...
  return(list(
    Z_B2 = Z_B2,                      # N x (K-1) - using DIFFERENCED psi
    Psi_theta_raw = Psi_theta_raw,      # N x T_total - beta_i * psi*_{i,t
       }
    diff_term_theta = diff_term_theta,  # N x 1 - beta_i * sum_s lambda_s
       psi*_{i,s}
    wt_sq_ind = wt_sq_ind,              # N x 1 - survey weights squared
    N = N, K = K, n = n,
    T_0 = T_0, T_total = T_total,
    B2 = B2,
    is_panel = TRUE
  ))
}

gen_hat_V_panel_fast <- function(precomp) {
  # Z_B2 already contains B2-transformed z_i using differenced psi
  Z_wt <- sqrt(precomp$wt_sq_ind) * precomp$Z_B2
  return((1 / precomp$n) * crossprod(Z_wt))
}
```

4 Asymptotic Variance of $\hat{\theta}_t(\lambda, w)$ under Repeated Cross-Sections

4.1 Theoretical object

In this case, we have that observations $(Y_{i,t}, G_{i,t})$ are independent across time, so the asymptotic variance of $\sqrt{n}(\hat{\theta}_t(\lambda, w) - \theta_t^*)$ can be expressed as follows

$$\begin{aligned}
\sigma_t^2(w) &= \mathrm{Var}(\psi_{it,\theta}(w)) \\
&= \mathrm{Var}\left(\psi_{i0,t} - \sum_{j=1}^{K} \psi_{ij,t} w_j\right) \\
&= \mathrm{Var}\left(\xi_{it}(w) - \sum_{s \in \mathcal{T}_0} \xi_{is}(w)\lambda_s\right) \\
&= \mathrm{Var}\left(\xi_{it}(w)\right) + \sum_{s \in \mathcal{T}_0} \mathrm{Var}\left(\xi_{is}(w)\right)\lambda_s^2 - 2 \times 1\{t \in \mathcal{T}_0\} \times \mathrm{Var}\left(\xi_{it}(w)\right)\lambda_t,
\end{aligned}$$

where $\xi_{it}(w) = \psi_{i0,t}^* - \sum_{j=1}^{K} \psi_{ij,t}^* w_j$. Plugging in the corresponding sample analog estimators of the variances yields the following estimator

$$\hat{\sigma}_t^2(w) = \frac{1}{n}\sum_{i \in N}\left(\hat{\xi}_{it}(w)\right)^2 + \sum_{s \in \mathcal{T}_0}\frac{1}{n}\sum_{i \in N}\left(\hat{\xi}_{is}(w)\right)^2 \lambda_s^2 - 2 \times 1\{t \in \mathcal{T}_0\} \times \frac{1}{n}\sum_{i \in N}\left(\hat{\xi}_{it}(w)\right)^2 \lambda_t, \tag{6}$$

where $\hat{\xi}_{it}(w) = \hat{\psi}_{i0,t}^* - \sum_{j=1}^{K} \hat{\psi}_{ij,t}^* w_j$.

4.2 Computational implementation

The function `gen_hat_sigma_squared_RC.R` returns a $T \times 1$ vector with all the estimates of $\sigma_t^2(w)$. In the code:

```
gen_hat_sigma_squared_RC <- function(w, data, lambda, group_means, n_jt, K
    , n, T_star, L) {

  T_0 <- T_star - 1
  T_total <- T_star + L

  # delta[t] = lambda[t] for t <= T_0, 0 otherwise
  delta <- c(lambda, rep(0, T_total - T_0))

  valid <- data$t >= 1 & data$t <= T_total
  G_idx <- data$G[valid] + 1L
  t_vals <- data$t[valid]
  lin_idx <- G_idx + (t_vals - 1L) * (K + 1L)

  # psi*_{i,t} = (n / n_jt) * (Y - m)
  psi_star <- (n / n_jt[lin_idx]) * (data$Y[valid] - group_means[lin_idx])

  # beta_i = 1 for treated (G=0), -w_k for donor k
  wt_mult <- ifelse(G_idx == 1L, 1, -w[G_idx - 1L])

  # (beta_i * psi*_{i,t})^2
```

```
  base2 <- (psi_star * wt_mult)^2
  wt_sq <- data$weight[valid]^2

  # sum_vec[t] = sum_{i in N_t} wt^2 * beta_i^2 * psi*^2
  sum_by_t <- tapply(wt_sq * base2, t_vals, sum, default = 0)
  sum_vec <- numeric(T_total)
  sum_vec[as.integer(names(sum_by_t))] <- sum_by_t

  # total_const = sum_s delta_s^2 * sum_vec[s] (cross-period contributions
     )
  total_const <- sum(wt_sq * base2 * delta[t_vals]^2)

  # sigma_t^2 = (1/n) * [total_const + (1 - 2*delta_t) * sum_vec[t]]
  # which equals (1/n) * [(1-delta_t)^2 * sum_vec[t] + sum_{s!=t} delta_s
     ^2 * sum_vec[s]]
  return(matrix((1/n) * (total_const + sum_vec * (1 - 2*delta)), ncol = 1)
     )
}
```

Here, the t-th component of the $T \times 1$ vector δ is equal to λ_t if $t \in \mathcal{T}_0$ and equal to zero otherwise.

5 Asymptotic Variance of $\hat{\theta}_t(\lambda, w)$ under Panel Data

5.1 Theoretical object

In this case, observations cannot be considered independent across time, so the estimator of $\sigma_t^2(w)$ takes the general form:

$$\hat{\sigma}_t^2(w) = \frac{1}{n} \sum_{i \in N} \hat{\psi}_{i,t,\theta}^2(w), \tag{7}$$

where $\hat{\psi}_{it,\theta}(w) = \hat{\psi}_{i0,t} - \sum_{j=1}^{K} \hat{\psi}_{ij,t} w_j$.

5.2 Computational implementation

The functions `precompute_panel_all.R` and `gen_hat_sigma_squared_panel_fast.R` compute $\hat{\sigma}_t^2(w)$ in two steps.

Step 1. First, the function `precompute_panel_all.R` computes all the $\hat{\psi}_{ij,t}$ components and returns them in a $N \times T$ matrix:

```
precompute_panel_all <- function(w, data, hat_Gamma_pre, lambda, B2,
                                 group_means, n_jt, K, n, T_star, T_total
                                    ) {
...
  # Compute components for psi_{i,t,theta} (sigma_squared)
  beta_lookup <- c(1, -w)
  beta_vec <- beta_lookup[G_by_id + 1L]

  # Psi_theta_raw stores beta_i * psi*_{i,t} before differencing
  Psi_theta_raw <- beta_vec * Psi_star_mat  # N x T_total

  # diff_term for theta = beta_i * sum_s lambda_s * psi*_{i,s}
  diff_term_theta <- beta_vec * diff_term_z  # N x 1
```

```
  # Return
  return(list(
    Z_B2 = Z_B2,                      # N x (K-1) - using DIFFERENCED psi
    Psi_theta_raw = Psi_theta_raw,      # N x T_total - beta_i * psi*_{i,t
       }
    diff_term_theta = diff_term_theta,  # N x 1 - beta_i * sum_s lambda_s
       psi*_{i,s}
    wt_sq_ind = wt_sq_ind,              # N x 1 - survey weights squared
    N = N, K = K, n = n,
    T_0 = T_0, T_total = T_total,
    B2 = B2,
    is_panel = TRUE
  ))
}
```

Step 2. Then, the function `gen_hat_sigma_squared_panel_fast.R` constructs the squares of the differenced components, takes the square of them, and return the sample average. In the code:

```
gen_hat_sigma_squared_panel_fast <- function(precomp) {
  # psi_{i,t,theta} = Psi_theta_raw[i,t] - diff_term_theta[i]
  #                 = beta_i * psi*_{i,t} - beta_i * sum_s lambda_s psi*_{
     i,s}
  #                 = beta_i * (psi*_{i,t} - sum_s lambda_s psi*_{i,s})

  Psi_theta <- precomp$Psi_theta_raw - precomp$diff_term_theta  # N x T_
     total
  return(matrix(
    colSums(precomp$wt_sq_ind * Psi_theta^2) / precomp$n,
    ncol = 1
  ))
}
```

6 Confidence Intervals for θ_t^* using Bonferroni Method

6.1 Theoretical object

To construct the 95% confidence intervals for the target parameter we use the following algorithm:

Algorithm 1 Computing Confidence Intervals for θ_t^*: Bonferroni Method

Input: $\hat{w}$, $\hat{\sigma}(\hat{w})$ and $\hat{V}(\hat{w})$.

1: Draw $w_1, \ldots, w_R \in \Delta_{K-1}$ i.i.d. from a distribution that has a full support on Δ_{K-1}.

2: Compute $T_{\mathsf{bf}}(w_r)$ and $\hat{c}_{1-\kappa,\mathsf{bf}}(w_r)$ with $\hat{V}(w)$ replaced by $\hat{V}(\hat{w})$ for each $r = 1, ..., R$.

3: Let

$$c_{U,R} = \max_{1 \le r \le R: T_{\mathsf{bf}}(w_r) \le \hat{c}_{1-\kappa,\mathsf{bf}}(w_r)} \left\{ \hat{\theta}_t(\lambda, w_r) + \frac{z_{1-\beta(\alpha,\kappa)} \hat{\sigma}_t(\hat{w})}{\sqrt{n}} \right\} \text{ and}$$

$$c_{L,R} = \min_{1 \le r \le R: T_{\mathsf{bf}}(w_r) \le \hat{c}_{1-\kappa,\mathsf{bf}}(w_r)} \left\{ \hat{\theta}_t(\lambda, w_r) - \frac{z_{1-\beta(\alpha,\kappa)} \hat{\sigma}_t(\hat{w})}{\sqrt{n}} \right\},$$

where $\beta(\alpha, \kappa) = (\alpha - \kappa)/2$.

Output: Confidence interval for θ_t^*:

$$C_{1-\alpha,R} = [c_{L,R}, c_{U,R}].$$

Here

$$T_{\mathsf{bf}}(w) = n(\hat{H}w - \hat{\boldsymbol{h}} - \hat{r}(w))' B_2 \hat{V}(\hat{w})^{-1} B_2' (\hat{H}w - \hat{\boldsymbol{h}} - \hat{r}(w)),$$

and $\hat{c}_{1-\kappa,\mathsf{bf}}(w)$ is the $(1-\kappa)$-th quantile of the χ^2 distribution with degrees of freedom equal to

$$\hat{k}(w) := \max\{K - 1 - \hat{d}(w), 1\},$$

where

$$\hat{r}(w) = \underset{r \ge 0,\, w'r = 0}{\arg\min} \; (\hat{H}w - \hat{\boldsymbol{h}} - r)' B_2 \hat{V}(\hat{w})^{-1} B_2' (\hat{H}w - \hat{\boldsymbol{h}} - r),$$

and

$$\hat{d}(w) = |\{j = 1, ..., K : \hat{\gamma}_j(w) = 0 \text{ and } w_j = 0\}|,$$

and

$$\hat{\gamma}(w) = B_2 \hat{V}(\hat{w})^{-1} B_2' (\hat{H}w - \hat{\boldsymbol{h}} - \hat{r}(w)).$$

6.2 Computational implementation

Step 1. Within the function `att_scd.R` we randomly draw the weights $w_1, ..., w_R$ using three different approaches. First, we draw 20 points evenly-spaced from the edges of the simplex Δ_{K-1}. Secondly, we draw a random sample of 500 points using the Dirichlet function `rdirichlet` in R. Lastly, we generate a cloud of 500 points around $\hat{w}$ in the simplex. Our final list of weights is the union of all of these subsets and the singleton $\{\hat{w}\}$. In the code:

```
...
  # Grid generation - put hat_w first for better early acceptance
  if (K > 1) {
    ij_pairs <- expand.grid(i = 1:(K - 1), j = 2:K)
    ij_pairs <- ij_pairs[ij_pairs$j > ij_pairs$i, ]
    edge_pts <- do.call(rbind, lapply(1:nrow(ij_pairs), function(k) {
```

```
      ij <- ij_pairs[k, ]; i <- ij$i; j <- ij$j
      v_i <- rep(0, K); v_i[i] <- 1; v_j <- rep(0, K); v_j[j] <- 1
      t(sapply(seq(0, 1, length.out = n_wpoints_per_edge), function(tt) tt
          * v_i + (1 - tt) * v_j))
    }))
  } else { edge_pts <- matrix(numeric(0), nrow = 0, ncol = K) }

  cloud_pts <- t(replicate(length_grid_w, {
    candidate <- hat_w + rnorm(K, 0, radius)
    u <- sort(candidate, decreasing = TRUE)
    cssv <- cumsum(u)
    rho <- max(which(u > (cssv - 1) / seq_along(u)))
    theta <- (cssv[rho] - 1) / rho
    pmax(candidate - theta, 0)
  }))

  # Put hat_w first in grid
  grid_w <- unique(rbind(hat_w, cloud_pts, gtools::rdirichlet(length_grid_
     w, rep(1, K)), edge_pts))
...
```

Step 2. The function `in_C_osqp.R` takes each w_r, computes $T_{\mathsf{bf}}(w_r)$ and $\hat{c}_{1-\kappa,\mathsf{bf}}(w_r)$, and evaluates whether $T_{\mathsf{bf}}(w_r) \leq \hat{c}_{1-\kappa,\mathsf{bf}}(w_r)$. To get $\hat{r}(w)$ we use the Operator Splitting Quadratic Program (OSQP). In the code:

```
in_C_osqp <- function(w, kappa, hat_H, hat_h, precomp, n, K, tolerance) {

  # Compute deviation
  phi_diff <- hat_H %*% w - hat_h

  if (all(w > 1e-8)) {
    # Interior point: r = 0 is optimal
    hat_r <- rep(0, K)
  } else {

    P_osqp <- 2 * precomp
    q_osqp <- -2 * as.vector(precomp %*% phi_diff)

    # Constraint matrix: first row is w' (equality), next K rows are I (r
       >= 0)
    A_osqp <- rbind(w, diag(K))
    l_osqp <- c(0, rep(0, K))
    u_osqp <- c(0, rep(Inf, K))

    # Convert to sparse matrix format for osqp
    P_sparse <- Matrix::Matrix(P_osqp, sparse = TRUE)
    A_sparse <- Matrix::Matrix(A_osqp, sparse = TRUE)

    # OSQP settings
    settings <- osqp::osqpSettings(
      verbose = FALSE,
      eps_abs = 1e-8,
      eps_rel = 1e-8,
```

```
      max_iter = 10000,
      polish = TRUE  # Improves solution accuracy
    )

    # Solve
    result <- tryCatch({
      model <- osqp::osqp(P = P_sparse, q = q_osqp, A = A_sparse,
                          l = l_osqp, u = u_osqp, pars = settings)
      model$Solve()
    }, error = function(e) {
      warning(paste("OSQP failed in in_C_osqp:", e$message))
      return(NULL)
    })

    if (is.null(result) || result$info$status_val != 1) {
      # OSQP failed or didn't converge - use fallback
      # Conservative: return FALSE (exclude from confidence set)
      return(FALSE)
    }

    hat_r <- result$x
    hat_r <- pmax(hat_r, 0)  # Numerical cleanup
  }

  # Compute test statistic
  diff_vec <- phi_diff - hat_r
  T_stat <- as.numeric(n * (t(diff_vec) %*% precomp %*% diff_vec))

  # Compute degrees of freedom using the A42 definition of gamma
  gamma_vec <- precomp %*% diff_vec


  # Find binding constraints: w is effectively 0 AND gamma is effectively
     0
  hat_d <- sum(abs(gamma_vec) < tolerance & w ==0)

  # Degrees of freedom (Note: K - 1 for the Bonferroni method per A42)
  hat_k <- max(K - 1 - hat_d, 1)

  # Critical value
  hat_c_1_kappa <- qchisq(1 - kappa, df = hat_k)

  return(T_stat <= hat_c_1_kappa)
}
```

Step 3. Lastly, inside the function `att_scd.R`, the candidates that satisfy the condition $T_{\mathsf{bf}}(w_r) \leq \hat{c}_{1-\kappa,\mathsf{bf}}(w_r)$ are used to obtain the lower and upper limits of the confidence interval. In the code:

```
...
  # Use OSQP-based in_C function
  in_C_func <- function(w) in_C_osqp(w, kappa, hat_H, hat_h, precomp_bf, n
     , K, tolerance)
```

```
  if (parallel) {
    parallel::clusterExport(cl, varlist = c("grid_w", "precomp_bf", "kappa
        ", "hat_H", "hat_h",
                                            "n", "K", "tolerance",
                                            "in_C_osqp"), envir =
                                               environment())
    parallel::clusterEvalQ(cl, library(osqp))
    parallel::clusterEvalQ(cl, library(Matrix))

    in_set <- unlist(parallel::parLapply(cl, seq_len(nrow(grid_w)),
        function(i) {
      in_C_osqp(as.numeric(grid_w[i, ]), kappa, hat_H, hat_h, precomp_bf,
          n, K, tolerance)
    }))
  } else {
    in_set <- unlist(lapply(seq_len(nrow(grid_w)), function(i) in_C_func(
        as.numeric(grid_w[i, ]))))
  }

  conf_set <- grid_w[in_set, , drop = FALSE]
  conf_set_size <- nrow(conf_set)
  z_val <- qnorm(1 - (alpha - kappa) / 2)

  # Determine which periods to compute
  if (post_treatment_only) {
    periods_to_compute <- (T_0 + 1):(T_0 + T_1)
  } else {
    periods_to_compute <- seq_len(T_0 + T_1)
  }

  for (tt in periods_to_compute) {
    # Degenerate case: variance = 0 (reference period with DID)
    if (hat_sigma_squared[tt] < 1e-12) {
      ci_matrix[tt, ] <- c(0, 0)
      next
    }

    if (tt <= T_0) {
      hat_mu_t <- hat_Gamma_pre[tt, ]; hat_mu_0t <- hat_gamma_pre[tt]
    } else {
      hat_mu_t <- hat_Gamma_post[tt - T_0, ]; hat_mu_0t <- hat_gamma_post[
          tt - T_0]
    }
    if (nrow(conf_set) == 0L) {
      ci_matrix[tt, ] <- NA_real_
    } else {
      synth_vals <- conf_set %*% hat_mu_t
      diffs <- hat_mu_0t - synth_vals
      se_tt <- sqrt(hat_sigma_squared[tt] / n)
      ci_matrix[tt, 1] <- min(diffs) - z_val * se_tt
      ci_matrix[tt, 2] <- max(diffs) + z_val * se_tt
    }
  }
...
```

Parallel processing is used via the `parallel` library in R to improve computational time.

7 Additional Tests

We test the finite-sample properties of the SCD estimator by pushing the design towards extreme cases of (n, K, T). The goal is to identify configurations where nominal 95% coverage breaks down, either because the number of observations per group becomes too small to support reliable inference, or because the dimensionality of the donor pool is large relatively to the cross-sectional dimension. We set $K \in \{5, 10, 25, 50\}$, pair each with a short and long time horizon, and vary $n \in \{50, 200, 1000\}$. We use the same data-generating process described in Scenarios A and B in the Monte Carlo simulations of Rincón and Song (2025). As a benchmark, we include the CS-DID estimator implemented in the `did` package (version 2.3.0; `https://github.com/bcallaway11/did`) of Callaway and Sant'Anna (2021), which is correctly specified under Scenario A but not under Scenario B. Results are based on $R = 1{,}000$ Monte Carlo iterations.

Entries marked "NA" indicate that SCD failed to converge because the number of observations per group is too small. CS-DID CI length under Scenario B is reported as "—" since DID cannot identify the target parameter in this case.

7.1 $K = 5$ Donors

Table 1: Coverage probability and CI length: $K = 5$, $T \in \{15, 55\}$, $n \in \{50, 200, 1000\}$.

		SCD Coverage		SCD CI Length		CS-DID Coverage		CS-DID CI Length	
n	T	A	B	A	B	A	B	A	B
50	15	0.996	0.984	5.634	3.750	0.972	0.631	4.147	—
	55	0.998	0.985	5.360	3.507	0.992	0.689	4.706	—
200	15	0.999	0.994	3.207	2.123	0.992	0.413	2.261	—
	55	0.999	0.996	2.969	1.964	0.995	0.451	2.540	—
1000	15	1.000	0.997	1.427	1.010	0.998	0.190	1.030	—
	55	1.000	0.995	1.292	0.924	0.999	0.220	1.166	—

7.2 $K = 10$ Donors

Table 2: Coverage probability and CI length: $K = 10$, $T \in \{20, 60\}$, $n \in \{50, 200, 1000\}$.

		SCD Coverage		SCD CI Length		CS-DID Coverage		CS-DID CI Length	
n	T	A	B	A	B	A	B	A	B
50	20	0.972	0.952	5.791	4.047	0.910	0.595	4.542	—
	60	0.972	0.950	5.408	3.788	0.941	0.690	5.199	—
200	20	1.000	0.995	4.140	2.779	0.993	0.471	2.778	—
	60	1.000	0.997	3.845	2.577	0.992	0.518	3.008	—
1000	20	1.000	0.999	1.940	1.327	0.994	0.236	1.293	—
	60	1.000	0.994	1.806	1.221	0.999	0.278	1.432	—

7.3 $K = 25$ Donors

Table 3: Coverage probability and CI length: $K = 25$, $T \in \{35, 75\}$, $n \in \{50, 200, 1000\}$.

		SCD Coverage		SCD CI Length		CS-DID Coverage		CS-DID CI Length	
n	T	A	B	A	B	A	B	A	B
50	35	NA	NA	NA	NA	0.786	0.526	4.327	—
	75	NA	NA	NA	NA	0.784	0.586	4.847	—
200	35	0.985	0.971	4.362	3.215	0.952	0.517	3.617	—
	75	0.982	0.972	3.630	2.836	0.958	0.571	3.938	—
1000	35	1.000	0.996	2.784	1.874	0.995	0.325	1.890	—
	75	1.000	0.998	2.629	1.710	0.998	0.343	2.052	—

7.4 $K = 50$ Donors

Table 4: Coverage probability and CI length: $K = 50$, $T \in \{60, 100\}$, $n \in \{50, 200, 1000\}$.

		SCD Coverage		SCD CI Length		CS-DID Coverage		CS-DID CI Length	
n	T	A	B	A	B	A	B	A	B
50	60	NA	NA	NA	NA	0.642	0.468	4.040	—
	100	NA	NA	NA	NA	0.695	0.491	4.317	—
200	60	0.986	0.923	3.469	1.904	0.863	0.574	4.517	—
	100	0.958	0.815	3.343	2.293	0.875	0.558	4.500	—
1000	60	1.000	0.991	2.625	1.842	0.990	0.447	2.571	—
	100	0.997	0.985	2.089	1.558	0.998	0.430	2.641	—

References

Callaway, B. and Sant'Anna, P. H. (2021). Difference-in-differences with multiple time periods. *Journal of Econometrics*, **225**, 200–230.

Rincón, R. and Song, K. (2025). Causal Inference with Groupwise Matching. *arXiv preprint*, **arXiv:2510.26106**.